

# Mock Test 3

Started: Aug 6 at 11:37am

## Quiz Instructions



### Question 1 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** Your system has operated with full human review for months. High confidence extractions appear highly accurate overall, and you want to automate those cases. Before deploying, what validation step is most critical?



Compare accuracy at different confidence thresholds to find an optimal cut off.



Analyze accuracy by document type and field to confirm high confidence performance is consistent across segments



Run a pilot that routes some high confidence extractions directly downstream and monitor reports.



Verify that aggregate accuracy meets downstream requirements.



### Question 2 15 pts

**Scenario: Claude Code for Continuous Integration** You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

**Question:** Your CI coordinator agent needs to spawn test-generation subagents scoped to write tests but not modify source code. How do you restrict the subagent?



Create a separate test-generation tool rather than a subagent



Add a system prompt instruction not to modify source code



Trust the agent to understand boundaries via instructions



Set allowedTools in the Task definition to include only Read, Write (for test files), and Grep



### Question 3 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your CI test generation system suggests test cases. How can you provide examples to improve consistency in test naming, structure, and coverage patterns?



Include several existing test examples in the prompt showing naming, structure, and comprehensive coverage



Describe test naming conventions and best practices in the prompt



Reference test generation guidelines in CLAUDE.md



Ask Claude Code to follow the project's testing style



### Question 4 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your CI pipeline includes Claude Code review comments on PRs. A developer has a PR with a known limitation flagged by the reviewer. On resubmission, the same issue is flagged again. What context should you provide to the CI reviewer?



Use a database to track which issues have been discussed



Skip review of known limitations to avoid duplicate comments



Ask developers to mark issues resolved before re-review



Include prior review findings and developer comments explaining the limitation is intentional



### Question 5 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your test generation CI job produces thousands of tests. Some are redundant with existing tests. What context would prevent this waste?

☐

Use a database to deduplicate generated tests

☐

Include existing test files in context so Claude Code avoids testing already-covered scenarios

☐

Run test generation in separate jobs to compartmentalize work

☐

Filter generated tests through a uniqueness checker



#### Question 6 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your CI code review has grown very large after analyzing many files and building context. The review is becoming unreliable with hallucinations. What should you do?

☐

Resume the existing session with --resume to preserve all analysis

☐

Clear the session cache and continue from the same point

☐

Increase the context window size to accommodate more information

☐

Start a fresh review with a summary of prior findings rather than continuing the stale session



#### Question 7 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your CI code review identifies security vulnerabilities. The criteria need to be very specific to be actionable. Instead of “check for security issues,” what should you specify?



“Flag when unvalidated user input is passed to SQL queries or shell commands” with concrete code examples



“Use security expertise to find vulnerabilities”



“Identify code that might have security problems”



“Rate code security and suggest improvements”



### Question 8 15 pts

**Scenario: Claude Code for Continuous Integration** You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

**Question:** You're designing a test case generation tool for your CI pipeline. The tool generates test cases for a given function. How should you describe this tool to help Claude Code select it appropriately?



A clear description explaining it generates test cases, what it accepts as input, and what test cases it returns



A description listing all possible test types it can generate



A vague description: “helps with testing”



A detailed implementation description of how the tool works internally



### Question 9 15 pts

**Scenario: Claude Code for Continuous Integration** You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

**Question:** Your CI pipeline includes a code quality gate that fails if security review flags are above a threshold. The system is currently very conservative, finding many low-severity issues. What's the best approach?



Disable the security gate entirely until criteria improve



Lower the severity threshold to pass with more issues



Define and enable only high-severity security checks; disable low-severity checks while refining criteria



Require manual approval for all security findings



### Question 10 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your team wants to use Claude Code for code review but worries about false positives creating noise. Which of these strategies is most effective for building trust?



Run preview mode first, then apply findings manually



Implement all possible checks immediately to maximize coverage



Use manual approval for all findings to ensure accuracy



Start with a small set of high-confidence checks, expand gradually as accuracy builds trust



### Question 11 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your CI pipeline runs Claude Code to review PRs. Some PRs span multiple domains (frontend, backend, database). Should you use specialized reviewers for each domain or one comprehensive reviewer?



Specialized reviewers per domain, each with domain-relevant tools and criteria, coordinated by a hub agent



One comprehensive reviewer analyzing all domains to ensure consistency



Sequential reviewers where frontend is reviewed first, then backend



Specialized reviewers without coordination to avoid bottlenecks



### Question 12 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your CI test generation is producing tests for paths that are already well-covered. Including existing test file content in context, how should you structure the prompt to avoid duplication?

☐

Show existing tests, explain coverage gaps, ask to generate tests for unexplored scenarios

☐

Include existing tests but expect the model to figure out gaps

☐

Show existing tests and ask to generate different tests

☐

Show test count and ask to generate more



Question 13 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your CI pipeline needs to analyze code from the perspective of two concerns: does this code work correctly, and does it follow security practices? What architecture would best handle this?

☐

Sequential analysis where functional correctness is checked first, then security

☐

Two agents running in the same session to share findings

☐

One comprehensive agent analyzing both concerns to ensure consistency

☐

Two independent agents focused on functional correctness and security respectively, coordinated by a hub



Question 14 15 pts

Scenario: Claude Code for Continuous Integration You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question: Your CI code review identifies potential bugs through pattern matching. Most findings are actionable, but some are false positives. What's the best approach to improve precision?



Use a voting system where multiple review instances must agree



Define specific, concrete patterns to check; disable patterns with persistent false positives while refining criteria



Require developer approval for all findings before posting



Lower the sensitivity threshold to reduce findings



### Question 15 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** Your pipeline uses an `extract_metadata` tool and also citation and DOI verification tools. When users ask for extraction and enrichment together, the model sometimes calls the enrichment tool first even though it needs metadata produced by `extract_metadata`. What is the most effective way to ensure structured metadata extraction happens first?



Set `tool_choice` to any and add prompt instructions prioritizing `extract_metadata`.



Force `extract_metadata` for every API call in the pipeline.



Force `extract_metadata` first, then process the enrichment request in subsequent turns after metadata is available.



Keep `tool_choice` as auto and reorder the tools array so `extract_metadata` appears first.



### Question 16 15 pts

**Scenario: Claude Code for Continuous Integration** You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

**Question:** Your CI pipeline coordinates multiple specialized Claude Code agents: one reviews tests, one reviews performance, one reviews security. What architectural pattern should you use?



Pipeline: sequential agents where each agent's output feeds the next



Consensus: all agents run independently and voting determines findings



Single agent with broad capabilities and role-based instructions



Hub-and-spoke: coordinator orchestrates specialized subagents with isolated context



### Question 17 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** Your extraction pipeline processes contracts with amendments. The model inconsistently extracts either the original term or the amended term, with no indication of which applies. What is the most effective approach?



Implement post extraction validation to detect amendments and flag those cases for review.



Redesign the schema so amended fields can capture multiple values, each with source location and effective date.



Add prompt instructions to always extract the most recent amendment value and ignore superseded terms.



Preprocess documents to remove superseded sections before extraction.



### Question 18 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** Your extraction model is hallucinating dates (creating dates that don't appear in the source document). How should you address this?



Accept the hallucinations; downstream systems can validate



Use validation to reject dates that don't match document patterns



Add a required\_evidence field to the schema; dates without source evidence are marked as hallucinations



Add a note in the system prompt: 'Do not hallucinate dates'



### Question 19 15 pts



**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** When validating extraction against a JSON schema, a document successfully validates but the extracted data is semantically wrong (e.g., a price is extracted but it's actually a phone number). How should you detect this?

☐

Use a separate model to verify semantic correctness

☐

Ask the user to review borderline extractions

☐

Add semantic validation rules: validate that prices are within reasonable ranges, emails contain @, phone numbers match regional patterns

☐

JSON schema is sufficient; semantic validation is downstream's problem



Question 20 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** Your system extracts information from multiple documents for the same entity (e.g., different invoices for the same customer). How should you handle discrepancies?

☐

Return an error if documents disagree

☐

Extract from all documents and include a record\_source field; downstream systems reconcile

☐

Use the first document's values

☐

Average values from multiple documents



Question 21 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Question: When extraction confidence is low for a critical field, what should the system do?



Extract with low confidence AND include an extraction\_quality flag; downstream systems decide if the extraction is usable



Return an error and skip the document



Extract the value anyway; confidence is just a number



Re-prompt the model to try again



Question 22 15 pts

Scenario: Structured Data Extraction You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Question: Your extraction system processes documents in a queue, and some fail. How should you structure error handling?



Return error and halt processing



Retry failed documents immediately



Skip failed documents and continue



For each document, track extraction success/failure and include failure reasons in output



Question 23 15 pts

Scenario: Structured Data Extraction You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Question: Before processing a large batch of documents, how should you validate your extraction system?



Skip testing; ML models handle variation automatically



Test the extraction on a sample of documents, review results manually, and refine the schema/prompts before bulk processing



Start processing; find issues as they arise



Run a single test extraction and assume scaling will work



#### Question 24 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** When designing extraction prompts, what pattern is most effective for reducing false positives and hallucinations?



Ask the model to explain its reasoning before extracting



Use only the schema; let the model figure out what to extract



Write a detailed prompt explaining what to extract



Provide explicit extraction criteria plus few-shot examples showing correct vs. incorrect extractions



#### Question 25 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** Your extraction system needs to extract nested data (e.g., invoice items with line-item details). How should the schema handle this?



Ask the user to specify nesting structure



Extract only top-level fields; nested data is too complex



Use nested objects in JSON schema: items (array of {product\_id, quantity, price}) with validation at each level



Flatten nested data into separate fields



#### Question 26 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** When extraction errors occur (model timeout, empty document, malformed response), what should you log?

☐

Log nothing; just skip the document

☐

Log the error and immediately retry

☐

Log full document content for debugging

☐

Log the error type, document identifier, extraction attempt count, and timestamp for debugging



Question 27 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** Your extraction system must maintain accuracy over time as documents evolve. What monitoring approach helps detect degradation?

☐

Sample extractions monthly and manually compare against prior results to detect drift

☐

Assume accuracy stays constant

☐

Track extraction confidence trends; if average confidence drops, investigate

☐

Re-test the full sample suite quarterly against current model



Question 28 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** When a field can have multiple valid formats (e.g., phone numbers as 1234567890, 123-456-7890, or +1 123-456-7890), how should your schema handle this?



Use a regex pattern that accepts all formats



Accept multiple formats and include a normalized version in output along with the original



Ask users to specify which format they want



Pick one format and normalize all inputs to that



### Question 29 15 pts

**Scenario: Structured Data Extraction** You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

**Question:** Your extraction system processes documents with optional fields (some have them, some don't). How should the schema define these?



Extract whatever is present; don't worry about missing fields



Include all fields as required; return error if any are missing



Mark optional fields in the schema; they can be null if genuinely missing from the document



Include optional fields with a default value



### Question 30 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

**Question:** Your agent generates boilerplate code files that should follow specific naming conventions and be placed in specific directories. How should you ensure correctness?



Generate files without validation; rely on users to verify



All of the above



Use Glob to check if files with correct naming exist, then verify via Read

- ☐ Use Bash to run linters or validation scripts that check naming and placement



### Question 31 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

**Question:** An engineer used Claude Code yesterday to investigate authentication flows in a monolith and wants to continue that specific named investigation today. How should she resume?

- ☐ Use resume with the specific session name.
- ☐ Start fresh and reread the same files.
- ☐ Use continue to pick up the most recent conversation.
- ☐ Use the session UUID manually from the transcript.



### Question 32 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

**Question:** Your product search tool frequently retries after zero results because the agent treats no matches found as a failure. The external API returns HTTP 200 with an empty results array. How should the tool's result be structured?

- ☐ Return a natural language string and let the agent interpret the outcome contextually.
- ☐ Return a structured result with a success boolean and results array, reserving error signaling for actual execution failures.
- ☐ Add a suggestions field with alternative search strategies when results are empty.
- ☐ Return isError true with a message that no products matched.



### Question 33 15 pts

Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question: Your log\_workout tool accepts exercise\_type, value, and measurement, and the agent frequently passes invalid combinations such as reps for cardio and miles for strength exercises. What would most effectively reduce these errors?

☐

Add enum constraints on the measurement field.

☐

Add explicit examples to the tool description showing valid combinations.

☐

Implement server side validation with descriptive error messages.

☐

Split the interface into log\_cardio\_workout and log\_strength\_workout with parameters appropriate to each category.



Question 34 15 pts

Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question: Your resource allocation tool returns only a simple acknowledgment after provisioning is requested. Users often confirm without understanding what they approved. What design change would most effectively address this?

☐

Add a detail\_level parameter controlling how much context the agent presents.

☐

Implement a hold period before execution completes.

☐

Return structured data including cost estimate, target project, resource specifications, and impact summary.

☐

Add a user\_acknowledged boolean parameter that must be set true



Question 35 15 pts

Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question: Your `remove_team_member` tool uses a `dry_run` parameter for previewing impacts before execution, but the agent sometimes bypasses the preview step. You need to ensure every removal is preceded by a preview that the user explicitly confirms. What is the most reliable approach?



Add server side validation that permits execution only if an identical preview occurred recently.



Add detailed instructions and few shot examples telling the agent to always call `dry_run` first.



Replace it with a two step flow: `preview_remove_member` returns impact details and a single use confirmation token, and `execute_remove_member` requires that token.



Annotate the tool as requiring confirmation and rely on the orchestration layer to prompt the user.



Question 36 15 pts

Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question: Your `update_game_score` tool accepts `game_date`, `home_team`, and `away_team`, but production logs show issues with nicknames, date formats, and selecting the wrong game when teams have rematches. What interface change would most effectively prevent these errors?



Replace the parameters with a `game_id` and a separate `search_games` lookup tool that returns matching IDs.



Add `season` and `confirm_before_update` parameters.



Add enum constraints for team names and a regex pattern for the date.



Add detailed examples showing the required date format and complete list of official team names.



Question 37 15 pts

Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question: Your `search_catalog` tool fails sometimes due to network timeouts that often succeed on retry and sometimes due to malformed user filters that never succeed on retry. Currently both are returned identically. How should the tool's error handling be modified?





Return all errors with a retryable boolean and error type details.



Implement automatic retry with backoff for network timeouts inside the tool and return syntax errors immediately with parameter validation details.



Add few shot examples to the system prompt explaining how to handle different error types.



Apply exponential backoff retries to all errors uniformly.



### Question 38 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

**Question:** Your `post_content` tool requires user confirmation before publishing, but the current confirmation step is so minimal that users approve almost instantly and miss errors such as wrong accounts or wrong schedule times. How should the confirmation workflow be redesigned?



Auto approve routine posts and require explicit confirmation only for unusual patterns.



Require users to type a confirmation phrase instead of clicking a button.



Add a mandatory waiting period before confirmation becomes available.



Include the complete post text, target account, scheduled time, and platform in the confirmation request.



### Question 39 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

**Question:** Your `control_device` tool manages smart home devices through external APIs. When a device does not respond within the timeout period, the tool returns an error. The agent currently tells users only that the device is not responding, without helpful next steps. Which error response structure would best enable useful follow up?



Return `is_error` true with a structured technical error containing only debugging fields such as device ID and raw API status.



Return `is_error` true with a message explaining the likely cause and suggesting troubleshooting steps the agent can offer.



Return `is_error` false with an optimistic message indicating the command may still be pending.



Return `is_error` true with a brief device offline message and make the agent call another tool for troubleshooting suggestions.



#### Question 40 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

**Question:** Your document analysis subagent has access to both a `'summarize_content'` and `'analyze_document'` tool. The model is frequently calling `'summarize_content'` when `'analyze_document'` would be more appropriate. Why is this happening and how do you fix it?



The tool descriptions are ambiguous. Expand them to clarify when to use each tool and provide example queries for each



The model is hallucinating; add error handling to reject `summarize_content` calls



Restrict the subagent to only the `analyze_document` tool using `allowedTools`



The subagent's system prompt doesn't mention `analyze_document`; add it explicitly



#### Question 41 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

**Question:** When your web search tool returns an empty result set, how should it indicate this to be distinguishable from an access error?



Return null or an empty array with `isError: false`



Return an error message indicating `'no results found'`

☐ Return a tool result with `isError: true` and `errorCategory` set to `'no_results_found'`

☐ Throw an exception that the subagent must catch



### Question 42 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

**Question:** Your coordinator wants to partition research work across subagents to parallelize efforts. Which partitioning strategy is most appropriate?

☐ Partition by processing phase: one subagent retrieves all sources, another analyzes all sources, a third synthesizes all findings

☐ Partition by source type: one subagent searches web results, another analyzes PDF documents, a third consults APIs

☐ Partition by output format: one subagent produces summaries, another produces detailed reports, a third produces data tables

☐ Partition by topic: each subagent researches a different subtopic of the main research question



### Question 43 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

**Question:** Your document analysis subagent works with potentially sensitive documents. What is the best practice for scoping access to minimize security risk?

☐ Define `allowedTools` to include only a vetted `'analyze_document'` tool with explicit read permissions and access logging

☐ Grant the subagent access to all document reading tools; let it decide what's appropriate

☐ Use MCP server scoping in `~/.claude.json` to restrict the subagent's MCP server connections to a document-specific server



Create a PostToolUse hook to redact sensitive fields before the subagent sees them



#### Question 44 15 pts

Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question: When defining tools for your subagents, you're deciding between making one 'search\_content' tool that handles web, documents, and APIs vs three specialized tools. What's the right choice?



Use one tool with conditional logic inside; agents will route based on parameters



Three specialized tools with clear, distinct descriptions reduce misrouting and allow role-specific optimization



Create both; let subagents choose which to use



One generalized tool reduces cognitive load and is easier to maintain



#### Question 45 15 pts

Scenario: Multi-Agent Research System You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question: Your research subagents each use different MCP servers (one for web APIs, one for document management). How should you configure these servers to ensure isolation and security?



Define project-wide servers in .claude/mcp.json; each subagent's AgentDefinition specifies which servers it can access



Define servers in ~/.claude.json for user-level access; subagents inherit the full set



Create separate environment configurations for each subagent



Define all servers in a single .mcp.json file in the project root; subagents filter at runtime



#### Question 46 15 pts

**Scenario: Multi-Agent Research System** You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

**Question:** Your expense reimbursement agent uses a `process_reimbursement` tool. Policy requires reimbursements above \$500 to be manager approved before funds are disbursed, and the threshold enforcement must be tamper proof. Which design ensures the \$500 threshold cannot be bypassed?



Use an `approved_by_manager` boolean parameter and tell the agent to set it only after approval.



Provide separate tools for auto reimburse and request manager approval, with prompt instructions telling the agent which to call.



Use a pre tool hook that inspects the amount and modifies the tool call to add a `requires_approval` flag.



Have `process_reimbursement` enforce the threshold internally, auto disbursing amounts at or below \$500 and creating a pending approval request for larger amounts.



#### Question 47 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

**Question:** You have configured an agent with multiple MCP servers for git, Jira, and documentation. When a user asks for a task that spans those systems, how does the agent access tools across the servers?



You must specify which server to use for each turn.



The agent queries each server sequentially to determine which handles each tool.



It selects the most relevant server and loads only that server's tools.



Tools from all configured MCP servers are discovered at connection time and are available simultaneously.



#### Question 48 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy

systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: Your agent needs to insert a helper function into the middle of a utility module, but the Edit tool cannot find unique text to match because the file contains repetitive patterns. What is the most reliable way to complete the insertion?



Use `replace_all` and embed the new function in the replacement text



Append the function at the end of the file using Bash



Use Edit with a very long `old_string` to try to force uniqueness



Use Read to load the file, insert the function in the right place, then Write the updated file



#### Question 49 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: A developer asks the agent to investigate why a specific API endpoint intermittently returns 500 errors. The codebase is large and the error must be traced through routing, middleware, business logic, and database layers. What task decomposition approach would be most effective?



Run parallel workers for all layers from the start and synthesize the results.



Dynamically generate investigation subtasks based on discoveries at each step and adapt the plan as new information emerges.



Create a comprehensive plan mapping all code paths before reading any files.



Use a fixed investigation sequence regardless of what intermediate findings show.



#### Question 50 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: After adding an MCP server with specialized code refactoring tools, the agent still uses basic text manipulation for refactoring tasks. The tools have minimal descriptions. What is the most

effective way to improve adoption of the specialized tools?

☐

Implement a classifier that automatically routes refactoring requests to the MCP server.

☐

Remove the Write tool so the agent must use MCP refactoring tools.

☐

Accept the behavior because simpler tools are more predictable.

☐

Enhance MCP tool descriptions to explain when each tool is preferable and clarify expected inputs and outputs.



Question 51 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: A codebase exploration tool stores session IDs so engineers can continue investigations. An engineer wants to resume a prior session, but some previously read files changed overnight. What approach best balances efficiency and accuracy?

☐

Resume without mentioning the changed files.

☐

Resume and inform the agent which specific files changed for targeted reanalysis.

☐

Resume without mentioning the changed files.

☐

Resume and immediately reread all previously analyzed files.



Question 52 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: You are building a security scanning workflow. When engineers need to locate all occurrences of a dangerous function such as eval across a large codebase, which tool should the agent use for content search?

☐

Use Bash with ls and grep on filenames.



Read the project's main entry file and follow imports.



Use Grep to search for the pattern across file contents.



Use Glob to find filenames matching the pattern and then read them.



### Question 53 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

**Question:** After integrating a local MCP server providing code analysis tools such as `analyze_dependencies`, the agent still uses Grep for dependency questions. The MCP tool description is very minimal. What is the most effective approach to improve MCP tool selection?



Remove Grep from the available tools when the MCP server is connected.



Add routing instructions to the system prompt telling the agent to use MCP tools for dependency questions.



Split `analyze_dependencies` into several granular tools.



Expand the MCP tool descriptions to detail capabilities and outputs such as direct imports, transitive dependencies, and cycles.



### Question 54 15 pts

**Scenario: Developer Productivity with Claude** You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

**Question:** Your codebase exploration agent is using the built-in tools effectively but users want to query specific knowledge (e.g., 'How does the payment system work?'). Should you create a custom MCP tool for domain knowledge?



No; ask users to provide domain knowledge in their initial prompt



Yes; create an MCP tool that connects to a documentation system or knowledge base





Yes; create an MCP tool that returns domain knowledge documents



No; the agent should explore the codebase using built-in tools



### Question 55 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: Your agent is tasked with refactoring a large function. What's the best approach?



Generate the refactored version, write it, then read it back to verify



Use a tool\_choice loop: first read and analyze, then propose changes to the user



Read the entire function, refactor in one go using Write



Use multiple small Edit operations to refactor step-by-step, testing after each change



### Question 56 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: Your agent is building a system that generates tests for a function. How should you structure this in the agentic loop?



Have the user specify test cases first, then generate tests based on their specifications



Use a test generation MCP server to generate tests automatically



Generate all tests at once and write them



Generate tests iteratively: create test 1, verify it runs, create test 2, verify, etc.



### Question 57 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: When your agent uses Bash to run tests, how should it handle test failures?

☐

Show the failure output to the user and ask them to fix it

☐

Ignore failures and assume the code works

☐

Use a tool\_choice: 'any' to force a tool call that understands the failure

☐

Parse the test output, identify the failure reason, and attempt to fix the code automatically



Question 58 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: Your agent is exploring a codebase written in an unfamiliar language. How should it discover coding patterns and conventions?

☐

Use an MCP server for language-specific knowledge

☐

Ask the user to explain conventions

☐

Use Glob and Grep to find example code (test files, well-documented functions) and analyze patterns through code examples

☐

Search for comments mentioning 'pattern' or 'convention'



Question 59 15 pts

Scenario: Developer Productivity with Claude You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question: Your productivity agent needs to automate repetitive code changes across multiple files (e.g., updating an API endpoint across 20 service files). What's the most reliable approach?

☐

Use Bash to find and replace across all files at once

☐

Use an MCP tool to apply batch changes

☐

Generate a script that makes the changes, then have the user run it

☐

Loop: for each file, Read it, Edit the content, verify with Read



### Question 60 15 pts

**Scenario: Claude Code for Continuous Integration** You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

**Question:** Your CI code review runs Claude Code in different configurations: strict for critical services, permissive for experimental code. Where should these different configurations live?

☐

In `.claude/rules/` with glob patterns for critical/ vs experimental/ directory paths

☐

In separate `CLAUDE.md` files at the service level

☐

In a master config file with if-then rules

☐

In `.claude/commands/` with conditional logic

---

Not saved

Submit Quiz